Algorithmic Spatial–Behavioral Types from Rewrite Systems

A Primer on OSLF, Native Type Theory, and Evidence-Valued Extensions

(Conceptual notes intended to accompany a Lean formalization snapshot; 2026-02-25)

Abstract

OSLF (Operational Semantics in Logical Form) proposes an algorithm that takes a language presented as a rewrite system—syntax with binders, structural equations, and reduction rules—and *mechanically generates* a spatial–behavioral logic and an associated type/judgment system. The construction is category-theoretic: starting from a category encoding syntax and dynamics, one passes through a presheaf topos via Yoneda, recovers internal Heyting structure from subobjects, and applies a Grothendieck construction to obtain a category of *native types*. NTT (Native Type Theory) describes a corresponding internal language with dependent types, logical connectives, and (in principle) inductive/coinductive fixed points.

This note distills the conceptual picture with an eye toward formalization and executability. In addition to the classical "types as predicates" reading, we highlight a *valuation layer*: OSLF propositions can be interpreted not only as truth values but also as evidence values (e.g. in a Heyting/quantale/fuzzy setting), enabling probabilistic, paraconsistent, or resource-aware extensions without changing the underlying operational core.

The document separates: (i) claims that are standard in the literature, (ii) endpoints that are already formalized in the accompanying Lean snapshot, and (iii) plausible research directions that are not yet theorems.

Status note. Implementation-status and count claims in this draft are snapshot-scoped; verify against current repository trackers and build outputs before citing as current state. Lean-proven claims are only those tied to explicit Lean file/theorem references; broader mathematical or historical claims are literature-backed context and not asserted as Lean-proven unless explicitly stated.

Contents

1 Why "logic from semantics"? 2

2 Input data: rewrite systems as "graph-structured theories" 3

2.1 Rewrite-system presentation . . . 3

2.2 From reductions to graph structure . . . 3

3 The OSLF construction in one diagram 3

3.1 The conceptual pipeline . . . 3

3.2 What is a native type? . . . 4

3.3 Modalities and a Galois connection . . . 4

4 Native Type Theory as internal language 4

4.1 Internal logic and dependent types . . . 4

4.2 Fixed points: μ/ν as inductive/coinductive types . . . 4

5 From truth to evidence: valuation layers **5**
5.1 Why evidence-valued semantics? . . . 5
5.2 Residuated/tensor structure . . . 5
5.3 Quantifiers as infimum/supremum . . . 5

6 Process calculi: π, ρ, and correspondence **5**
6.1 Why correspondence matters . . . 5
6.2 Derived vs core semantics . . . 5
6.3 Substitutability as a "crown" statement . . . 6

7 Linguistic and neural connections (speculative but testable) **6**
7.1 Operational cores of grammar . . . 6
7.2 Tensorization and distributional semantics . . . 6

8 Lean formalization snapshot: what is already there **6**
8.1 Executable reduction and soundness scaffolding . . . 6
8.2 Language-parametric generation . . . 6
8.3 Categorical bridge endpoints . . . 7
8.4 Evidence/tensor endpoints . . . 7

9 Claim hygiene and open problems **7**
9.1 What is standard . . . 7
9.2 What is formalized (snapshot-dependent) . . . 7
9.3 What remains genuinely open . . . 7

A Appendix: suggested code-map anchors **7**

B Appendix: a minimal glossary **8**

1 Why "logic from semantics"?

The usual workflow in programming languages is: define a calculus, then separately design (or port) a logic and type system to reason about it. OSLF reverses the order of invention: given a machine-readable description of a model of computation as a rewrite system, it aims to *derive* a spatial–behavioral logic and type discipline by a uniform recipe.

Two guiding intuitions:

(1) **Types as queries over behavior.** A type is not merely a shape constraint on syntax; it is a property that can talk about structure (*spatial* connectives) and dynamics (*behavioral/modal* connectives).

(2) **Witnesses matter.** Under Curry–Howard and realizability-style readings, semantics of formulae are collections of witnesses. Sets are one option, but not the only option; changing the witness-collection algebra changes the induced logical connectives.

For concurrency, the intended payoff is a uniform method to generate logics that can express liveness/safety properties and structural facts (e.g. parallel composition) that are invisible to many traditional modal logics.

2 Input data: rewrite systems as "graph-structured theories"

2.1 Rewrite-system presentation

An OSLF input language can be thought of as:

- **Syntax with binders:** term constructors, potentially higher-order, described using a higher-order abstract syntax (HOAS) discipline.
- **Equations:** a structural congruence/definitional equality that quotients raw terms.
- **Reduction rules:** a (possibly labeled) transition relation generated from rules, often context-closed and compatible with equations.

In the ρ-calculus and related calculi, the mutual recursion between *names* and *processes*, and the associated quote/drop structure, is a canonical example: names can be built from processes, and processes can execute names.

2.2 From reductions to graph structure

A recurring theme in OSLF and related work is to represent operational semantics internally as a *graph structure* whose edges witness allowed transitions. This is closely aligned with the view that operational semantics correspond to GSOS-style rules and can be seen categorically as distributive laws.

3 The OSLF construction in one diagram

3.1 The conceptual pipeline

A compact high-level view is:

$$\text{(rewrite system)} \rightsquigarrow \underbrace{\text{(category } \mathcal{C} \text{ of syntax/constructors)}}_{\text{“constructor category”}} \xrightarrow{\text{Yoneda}} \widehat{\mathcal{C}} = \mathbf{Set}^{\mathcal{C}^{op}} \rightsquigarrow \text{(subobjects} \Rightarrow \text{Heyting structu}$$

In prose:

(1) Start from a category $\mathcal{C}$ that encodes the language's sorts and (crucially) the *sort-crossing constructors* that move between sorts (e.g. quote/drop).

(2) Apply Yoneda to move into a presheaf setting, where each object has a representable presheaf.

(3) Interpret predicates as subobjects (subfunctors) of representables; the poset of subobjects carries Heyting-algebra structure (internal logic).

(4) Apply the Grothendieck construction to obtain a category whose objects can be read as pairs (X, U): a *sort* X together with a *predicate/filter* U on its representable, i.e. a native type.

3.2 What is a native type?

A useful operational reading is:

> A *native type* is a *sort* together with a *behavioral predicate* over terms/patterns of that sort.

In the Lean snapshot, this "type = (sort, predicate)" idea appears explicitly in the generated-typing layer.

3.3 Modalities and a Galois connection

A distinctive feature of OSLF is that modal operators are not bolted on: they are *generated*. In a two-sort calculus like the ρ-calculus, sort-crossing constructors provide canonical candidates for modal operators. Abstractly, one gets an adjunction (often presented as a Galois connection) $\Diamond \dashv \Box$ linking possibility and necessity.

Conceptually:

- $\Diamond\varphi$ expresses "there exists a behavior step (or path) after which φ holds".
- $\Box\varphi$ expresses "for all behavior steps (or predecessors, depending on polarity), φ holds".

The key point is structural: the adjunction is derived from the same categorical data used to describe syntax and operational edges.

4 Native Type Theory as internal language

4.1 Internal logic and dependent types

NTT describes the internal language of the native-type construction, including:

- propositional connectives (interpreted in a Heyting setting);
- dependent sums Σ and products Π, subsuming existential and universal quantification;
- comprehension types $\{x : A \mid \varphi\}$ connecting predicates and types.

The conceptual advantage is that once the presheaf/topos infrastructure is in place, many logical constructions come from general categorical principles rather than ad hoc encodings.

4.2 Fixed points: μ/ν as inductive/coinductive types

A longer-term goal is to connect operational modal reasoning to a μ-calculus style fixed-point logic:

- μ for least fixed points (inductive properties like reachability);
- ν for greatest fixed points (coinductive properties like invariants/bisimulation).

This is where Knaster–Tarski style fixed-point constructions in an appropriate fibration meet the operational layer.

5 From truth to evidence: valuation layers

5.1 Why evidence-valued semantics?

In many applications (world modeling, uncertain inference, inconsistent data), one wants a semantics more refined than $\{\texttt{true}, \texttt{false}\}$. One can keep the OSLF operational core (the transition system) and change only the *codomain* of formula evaluation:

$$[\![\varphi]\!](p) \in V$$

where V is a partially ordered algebra of evidence/weights rather than $\mathbb{B}$.

This makes the "collection of witnesses" parameter explicit.

5.2 Residuated/tensor structure

A common pattern is to endow V with a monoidal tensor $\otimes$ and a residuation (implication-like operator) $\Rightarrow$, yielding resource-aware entailment:

$$a \otimes b \leq c \quad \Longleftrightarrow \quad b \leq (a \Rightarrow c).$$

In a fuzzy/probabilistic instance, one can take $V = [0, 1]$ with a chosen t-norm as $\otimes$. In a paraconsistent instance, V may track positive/negative support separately.

5.3 Quantifiers as infimum/supremum

When quantifiers are added, a clean semantics in a complete lattice is:

$$[\![\forall x.\, \varphi]\!] = \bigwedge_{d \in \mathrm{Dom}} [\![\varphi[x := d]]\!], \qquad [\![\exists x.\, \varphi]\!] = \bigvee_{d \in \mathrm{Dom}} [\![\varphi[x := d]]\!].$$

This mirrors the Π/Σ view in NTT, and it is compatible with evidence lattices beyond Booleans.

6 Process calculi: π, ρ, and correspondence

6.1 Why correspondence matters

Completeness and expressiveness results for modal/spatial logics are often proved *relative to* an operational notion of equivalence, such as (weak) bisimulation or barbed congruence. To transfer results between calculi (e.g. from π-calculus to ρ-calculus), one typically needs a high-quality encoding and an operational correspondence theorem: forward simulation and some form of backward reflection.

6.2 Derived vs core semantics

A recurring engineering issue in formalizations is separating:

- **core/canonical reduction:** the intended semantics of the calculus;
- **derived/administrative steps:** convenience rules (e.g. unfolding replication, name admin) that should not change observable behavior.

A robust architecture proves correspondence in the derived layer (because proofs are shorter), then transports results back to the core via conservativity/adequacy theorems on a well-defined fragment.

6.3 Substitutability as a "crown" statement

A central claim in the OSLF narrative is a substitutability theorem: operational equivalence (bisimilarity) coincides with logical equivalence in the generated type logic. Such theorems are often in the spirit of Hennessy–Milner results, but strengthened by spatial operators that can observe structure.

7 Linguistic and neural connections (speculative but testable)

This note also records a line of thought motivated by grammar formalisms and neural language models.

7.1 Operational cores of grammar

One can view a grammar formalism as a rewrite system: it generates structures and admits reductions/derivations. If multiple natural-language grammars are treated as instances, one can ask for a *weakest operational core* that preserves a shared logical interface (what can be expressed/queried).

This echoes "minimalism" intuitions in linguistics, but the formal content is: *identify a minimal rewrite kernel that conservatively preserves the induced spatial–behavioral logic.*

7.2 Tensorization and distributional semantics

A separate (and still open) research direction is to compile logical/operational forms into tensorial expressions (using a tensor logic or related categorical semantics), thereby connecting symbolic derivations to distributional/statistical models.

A pragmatic hypothesis for transformer-style architectures is that long-term memory need not store complete sequence detail; in many tasks it suffices to store a structured partial order or summary that supports the queries of interest. This is naturally phrased using order/enrichment (lattices/quantales) rather than raw lists.

8 Lean formalization snapshot: what is already there

The accompanying Lean snapshot (in a `mettapedia` project) contains a significant amount of executable and theorem-proved infrastructure. This section only lists *representative* anchor points, without attempting to be complete.

8.1 Executable reduction and soundness scaffolding

A key milestone for usability is having an executable one-step reducer for a core calculus with a proven soundness theorem, plus regression tests.

8.2 Language-parametric generation

The formalization factors out a `LanguageDef` describing a user-defined language (constructors + rewrite rules), then mechanically builds:

- a generated native-type notion (`GenNativeType`);
- a generated typing judgment (`GenHasType`);
- a substitutability-style stability story (types preserved under substitution).

8.3 Categorical bridge endpoints

The snapshot also includes explicit endpoints that tie the categorical constructions to concrete modal operators and internal logic packages.

8.4 Evidence/tensor endpoints

Finally, there are small bridge theorems for evidence/tensor structure (e.g. residuation on $[0, 1]$) intended to make it easy to connect the logic layer to quantitative interpretations.

9 Claim hygiene and open problems

9.1 What is standard

The following are standard (non-controversial) mathematical facts used by the program:

- Presheaf categories are toposes and support an internal Heyting logic.
- Subobjects of a fixed object form a Heyting algebra (and often a complete lattice under mild conditions).
- Grothendieck constructions package indexed families (fibrations) into total categories.

9.2 What is formalized (snapshot-dependent)

The Lean snapshot provides concrete instantiations of: generated modal operators, Galois connections, internal-language bridge packages, and executable reduction/matching infrastructure. (See the module pointers in the appendix.)

9.3 What remains genuinely open

Some ambitious directions are not "for free" and should be treated as research problems:

- **Generic completeness of spatial logics.** Even if modalities are generated functorially, completeness (in the sense of "logical equivalence implies bisimulation") typically demands image-finiteness or similar hypotheses.
- **Transport of completeness across encodings.** Transferring completeness from one calculus to another requires encodings that preserve the relevant observation structure and adequate reflection properties.
- **Fixed points + operational distributive laws.** It is plausible that distributive-law structure helps organize proofs, but a true "completeness-for-free" theorem would need very careful statement and nontrivial conditions.

A Appendix: suggested code-map anchors

The following filenames/theorem identifiers are useful anchors when navigating the Lean snapshot:

- `Mettapedia/OSLF/Framework/GeneratedTyping.lean` — generated typing pipeline.

- `Mettapedia/OSLF/Framework/CategoryBridge.lean` — modal adjunction endpoints.
- `Mettapedia/OSLF/Framework/ToposTOGLBridge.lean` — topos/internal-language bridge packages.
- `Mettapedia/OSLF/QuantifiedFormula.lean` — quantified formula extension and inf/sup semantics.
- `Mettapedia/Languages/ProcessCalculi/*` — ρ/π calculi, encodings, bisimulation layers.

B Appendix: a minimal glossary

Rewrite system Syntax + equations + reduction rules.

Spatial logic Logic with connectives that observe structure (e.g. parallel composition).

Behavioral logic Modal logic observing operational steps (e.g. possibility/necessity).

Presheaf topos Category of contravariant set-valued functors $\mathbf{Set}^{\mathcal{C}^{op}}$.

Heyting algebra Lattice with implication appropriate for intuitionistic logic.

Quantale Complete lattice with monoidal product distributing over joins (resource/evidence models).

Galois connection Adjunction between posets; here used for $\Diamond \dashv \Box$.

Acknowledgment of sources: This primer is a synthesis of standard category-theoretic semantics, the OSLF and NTT papers, and a Lean formalization snapshot in the `mettapedia` codebase.